

📄 Résumé de la faille

Chaîne d'exploitation complète :

- 1. Le site accepte un compte avec **identifiants vides** (username + password vides) → accès sans credentials
- 2. La page cachée `/sessions` expose toutes les sessions actives, dont celle de l'**admin**
- 3. En remplaçant mon cookie par celui de l'admin → je prends son identité **sans mot de passe** → flag

Failles combinées : authentification défaillante (champs vides acceptés) + fuite d'information (`/sessions`) + session hijacking.

🔍 Le mécanisme exact (confirmé via Network / HAR)

Ce que le navigateur a **réellement** envoyé :

```
POST /register → username=&password=&conf_password=
                (CHAMPS VIDES) → 302, redirige vers /login

POST /login → username=&password=
                (CHAMPS VIDES) → 302, redirige vers /
```

Le serveur a **accepté des identifiants vides** et m'a connecté. C'est pourquoi « register, register, login » a marché sans rien taper : je n'avais pas besoin d'identifiants.

Cohérence avec `/sessions` :

```
session:XXXX {'key': 'admin'} ← admin
session:YYYY {'key': ''}      ← MOI (compte à username vide)
```

Le `key` vide = mon compte sans nom. Tout colle.

🔧 Méthodo — les étapes

- 1. **Lire l'énoncé** → « session qui n'expire jamais », attaquant qui réutilise une session. Indice : faille de **gestion de session**.
- 2. **Accéder au site** → register + login avec **champs vides** : accepté (faille d'auth) → session lambda valide (`key: ''`).
- 3. **Inspecter le cookie** (F12 → Application → Cookies) → cookie `session` avec valeur encodée.
- 4. **Tester le déchiffrement** (fausse piste — à éliminer vite) : - `flask-unsign` → « pas un cookie Flask » - CyberChef Magic → binaire chiffré illisible - **Conclusion** : cookie chiffré côté serveur, pas la voie. Ne pas s'acharner sur la crypto.
- 5. **Lire le contenu du site** (le bon réflexe) → les commentaires de la page d'accueil : « *I found a strange page at /sessions* » → indice caché dans le contenu.
- 6. **Aller sur** `/sessions` → toutes les sessions actives exposées en clair, dont le cookie de l'admin (`key: 'admin'`).
- 7. **Voler la session admin** (hijacking) → Application → Cookies → remplacer la valeur de mon cookie `session` par celle de l'admin → F5 → **FLAG**.

✅ Ce que j'aurais dû regarder pour trouver les pistes seul

Checklist web à faire **systématiquement** au début :

- [] Lire l'énoncé + **tous** les hints
- [] **Tester les entrées** : champs vides ? valeurs bizarres ? (*ici : champs vides acceptés*)
- [] Regarder le **code source** (Ctrl+U)
- [] Lire les **commentaires** (HTML et contenu utilisateur) → l'indice `/sessions` était là
- [] Inspecter les **cookies** (F12 → Application)
- [] Onglet **Network** (+ Preserve log) : voir ce qui est **réellement envoyé/reçu** → l'outil qui a résolu le « j'ai réussi sans savoir comment »
- [] Tester les **endpoints cachés** : `/sessions` `/admin` `/robots.txt`

Règles d'or : - Le contenu du site (commentaires, pages, source) contient presque toujours les indices. - La crypto/déchiffrement est souvent une **fausse piste**. - Quand « ça marche sans comprendre » → onglet **Network** pour voir la vérité brute.

🔧 Outils utilisés

Outil	Usage
DevTools → Application → Cookies	Voir et modifier les cookies
DevTools → Network (+ Preserve log)	Voir les requêtes réelles (payloads, status)
Export HAR	<code>Save all as HAR</code> — ⚠️ contient cookies et mots de passe en clair. CTF = OK à partager, site réel = jamais sans nettoyage
CyberChef (Magic)	Tester les encodages
flask-unsign (via pipx)	Tester si cookie Flask

🛡️ Concept : Session Hijacking

Un **cookie de session** = ta carte d'accès une fois connecté. Voler le cookie de session de quelqu'un = prendre son identité **sans mot de passe**, tant que la session est valide.

Dans la vraie vie, le vol de session se fait par : - **XSS** (script qui vole le cookie de la victime) - **Session exposée** (comme ici, `/sessions`) - **Interception** réseau non chiffré - **Session qui n'expire jamais** (l'énoncé)

Protections côté serveur : cookies `HttpOnly` + `Secure`, expiration correcte des sessions, ne jamais exposer les sessions, régénérer l'ID de session à la connexion, et **valider les entrées** (pas de compte à identifiants vides !).